

Top 100 .NET Interview Questions

For a Senior / 10+ Years Experienced Candidate

C# Language & .NET Core Fundamentals

1. Explain the differences between value types and reference types, and how boxing/unboxing affects performance.
2. What are the differences between IEnumerable, IEnumerator, IQueryable, and IAsyncEnumerable?
3. Explain deferred execution in LINQ and how it can lead to subtle bugs.
4. What is the difference between ref, out, and in parameters?
5. Explain the differences between abstract classes and interfaces, including default interface methods in C# 8+.
6. How does the .NET garbage collector work? Explain generations 0, 1, and 2.
7. What is the difference between Finalize, Dispose, and the using statement / IDisposable pattern?
8. Explain record types in C# and when to use them over classes.
9. What is pattern matching and how have switch expressions evolved in modern C#?
10. Explain nullable reference types and how they help prevent NullReferenceExceptions.
11. What is the difference between const, readonly, and static readonly?
12. Explain extension methods and their limitations.
13. What are delegates, multicast delegates, and how do events build on them?
14. Explain the differences between Func, Action, and Predicate delegates.
15. What is reflection, and what are its performance implications?
16. Explain expression trees and a real-world use case.
17. What is the difference between shallow copy and deep copy in C#?
18. Explain span and memory and why they matter for performance.
19. What are tuples and ValueTuples, and how do they differ?
20. Explain the differences between String, StringBuilder, and string interpolation performance.

Asynchronous Programming & Multithreading

1. Explain the differences between Task, Task, and ValueTask.
2. What is the difference between async/await and the Task Parallel Library (TPL)?
3. Explain SynchronizationContext and why ConfigureAwait(false) matters.
4. What is a deadlock in async code, and how can it occur with .Result or .Wait()?
5. Explain the differences between Thread, ThreadPool, and Task.
6. What is the difference between lock, Monitor, Mutex, and Semaphore/SemaphoreSlim?
7. How do you handle cancellation in async operations using CancellationToken?
8. Explain the producer-consumer pattern using BlockingCollection or Channels.
9. What are race conditions, and how do you detect and prevent them?

10. Explain the differences between `Parallel.ForEach` and `PLINQ`.
11. What is the `volatile` keyword and when is it needed?
12. Explain async streams (`IAsyncEnumerable`) and their use cases.

ASP.NET Core & Web APIs

1. Explain the ASP.NET Core middleware pipeline and how request processing flows through it.
2. What is the difference between `AddScoped`, `AddTransient`, and `AddSingleton` in dependency injection?
3. Explain how model binding and model validation work in ASP.NET Core.
4. What is the difference between Minimal APIs and traditional Controller-based APIs?
5. Explain how authentication and authorization differ, and describe JWT-based authentication flow.
6. What is CORS, and how do you configure it securely in ASP.NET Core?
7. Explain action filters, exception filters, and middleware - when would you use each?
8. How does ASP.NET Core handle configuration (appsettings, environment variables, secrets)?
9. Explain the Options pattern (`IOptions`, `IOptionsSnapshot`, `IOptionsMonitor`).
10. What is the difference between server-side rendering, Blazor Server, and Blazor WebAssembly?
11. How do you implement API versioning in ASP.NET Core?
12. Explain how health checks work and why they matter in microservices.
13. What is the difference between synchronous and asynchronous controller actions, and when does it matter?
14. Explain how output caching and response caching work.
15. How would you secure a Web API against common attacks (XSS, CSRF, SQL Injection, over-posting)?

Entity Framework Core & Data Access

1. Explain the differences between eager loading, lazy loading, and explicit loading in EF Core.
2. What is the N+1 query problem, and how do you avoid it?
3. Explain change tracking in EF Core and how `AsNoTracking` improves performance.
4. What is the difference between Code First, Database First, and migrations?
5. How do you handle concurrency conflicts in EF Core (optimistic concurrency)?
6. Explain the Unit of Work and Repository patterns - are they still relevant with EF Core?
7. What are compiled queries in EF Core, and when should you use them?
8. How do you handle database transactions spanning multiple operations in EF Core?
9. Explain the differences between `IQueryable` and `IEnumerable` when querying with EF Core.
10. How do you optimize EF Core for bulk insert/update/delete operations?
11. What is the difference between owned entities, value objects, and shadow properties in EF Core?
12. How would you implement soft deletes and global query filters in EF Core?

Architecture, Design Patterns & SOLID

1. Explain each of the SOLID principles with a practical example.
2. What is Dependency Injection vs Inversion of Control vs Dependency Inversion?
3. Explain the differences between Repository, Unit of Work, CQRS, and Mediator patterns.
4. What is the difference between monolithic, microservices, and modular monolith architectures?
5. Explain the Circuit Breaker pattern and how Polly implements it.
6. What is the Mediator pattern, and how does MediatR implement it in .NET?
7. Explain Domain-Driven Design concepts: aggregates, entities, value objects, and bounded contexts.
8. What is event sourcing, and how does it differ from traditional CRUD persistence?
9. Explain the Saga pattern for distributed transactions in microservices.
10. What is the difference between synchronous and asynchronous (message-based) communication between microservices?
11. Explain the Strategy, Factory, Decorator, and Adapter patterns with .NET examples.
12. How do you design for idempotency in distributed systems?
13. What is the API Gateway pattern, and what problems does it solve?
14. Explain caching strategies: cache-aside, write-through, write-behind, and where Redis fits in.

Performance, Testing & DevOps

1. How do you profile and diagnose memory leaks in a .NET application?
2. Explain server garbage collection vs workstation garbage collection, and when to use each.
3. What tools would you use to diagnose high CPU usage in production (dotnet-trace, dotnet-counters)?
4. Explain the differences between unit tests, integration tests, and end-to-end tests, and how you structure them in .NET.
5. What is mocking, and how do frameworks like Moq or NSubstitute help in unit testing?
6. Explain Test-Driven Development (TDD) and its benefits/drawbacks.
7. How do you implement CI/CD pipelines for a .NET application (build, test, deploy stages)?
8. What is containerization, and how do you dockerize a .NET application?
9. Explain structured logging and the role of Serilog/ILogger with correlation IDs.
10. How do you implement distributed tracing and observability (OpenTelemetry) in microservices?
11. What strategies would you use to reduce application startup time and cold-start latency?
12. Explain how you would design a rate-limiting mechanism for an API.
13. What is the difference between vertical and horizontal scaling, and how does it affect application design?
14. How do you handle configuration and secrets management securely across environments (e.g., Azure Key Vault)?

Database, Security & Miscellaneous

1. Explain ACID properties and how they relate to database transaction isolation levels.
2. What is the difference between optimistic and pessimistic locking in databases?
3. How do you prevent SQL injection in raw SQL queries executed from .NET?
4. Explain the differences between symmetric and asymmetric encryption, and where each is used in .NET apps.
5. What is OAuth 2.0 vs OpenID Connect, and how do they relate to ASP.NET Core Identity?
6. Explain how you would design a multi-tenant application architecture in .NET.
7. What is the difference between message queues (RabbitMQ/Azure Service Bus) and event streaming (Kafka)?
8. How do you ensure backward compatibility when evolving a public API?
9. Explain the differences between .NET Framework, .NET Core, and .NET 5+ (unified .NET).
10. What is AOT (Ahead-of-Time) compilation in .NET, and what are its benefits and trade-offs?